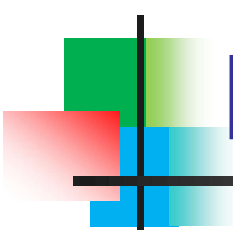


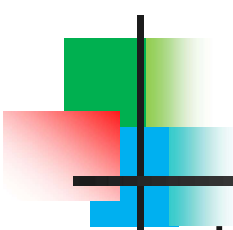
Pointers

Pointer arithmetic



Review

- What are pointers?
- Primary operators used in pointers
- Why pointers?
- Types of pointers
 - void pointer
 -



Adding pointers

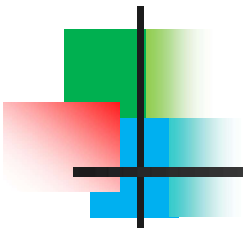
- `int num = 5, *p = &num, *q;`
- Construct a pointer to the next integer after `*p` and assign it to `q`

`q = p + 1;`

This means address of `q` is greater than `p` of 4 bytes because both in integer. If $\text{add}(p) = 1000$, then $\text{add}(q) = 1004$.

- The amount added to the pointer is multiplied by the size of the type the pointer points to

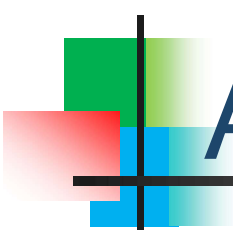




Add/subtract pointer

- `long int *p, *q;`
`p++; q--;`
 - Increment p to point to the next long int;
decrement q to point to the previous long int
- C never checks that the resulting pointer is valid

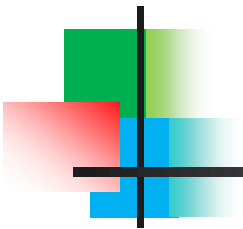




Add n integers to a pointer

- `double *p, *r; int n = 4;`
- Construct a pointer to a double that is n doubles beyond *p, and assign it to r
 - `r = p + n;`
- n may be negative





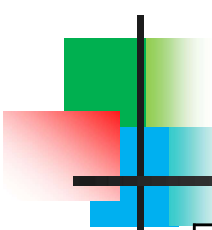
Subtract pointers

- `float *p, *q; int n;`

`n = p - q;`

- `n` is the number of floats between `*p` and `*q`;
i.e., what would be added to `q` to get `p`



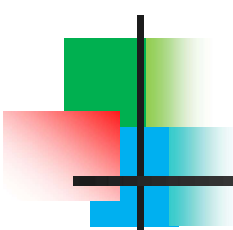


Pointer arithmetic

<type> *p	Pointer addition	sizeof(*p)	Bytes added
char	$p + 1$	1	1
short	$p + 1$	2	2
int	$p + 1$	4	4
float	$p + 1$	4	4
char	$p + 2$	1	2
short	$p + 2$	2	4
int	$p + 2$	4	8
float	$p + 2$	4	8

Pointer arithmetic is meaningless unless performed on an array.





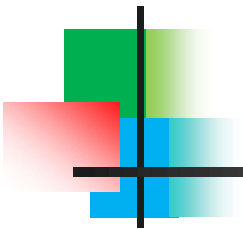
Arrays and Pointers

- Arrays and pointers are *closely related*
- They are essentially the same thing!
 - Esp. when used as parameters of functions

```
int A[10], *p;
```

```
p = A; // legal assignment
```





Printing arrays using pointers

- ✎ Write a C program to print the elements of an array using pointers.

Ex: pointers and arrays

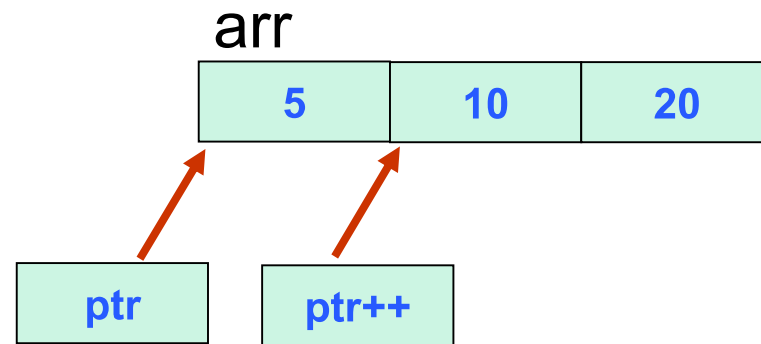
```
int arr[3]= { 5, 10, 20 };
```

```
int *ptr ;
```

```
ptr = arr ;
```

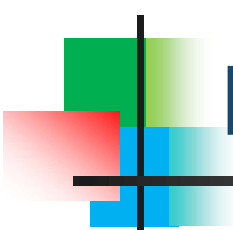
```
printf("%d\t",*ptr);
```

```
printf("%d\t",*ptr++);
```



How about `ptr = &arr[0]`??

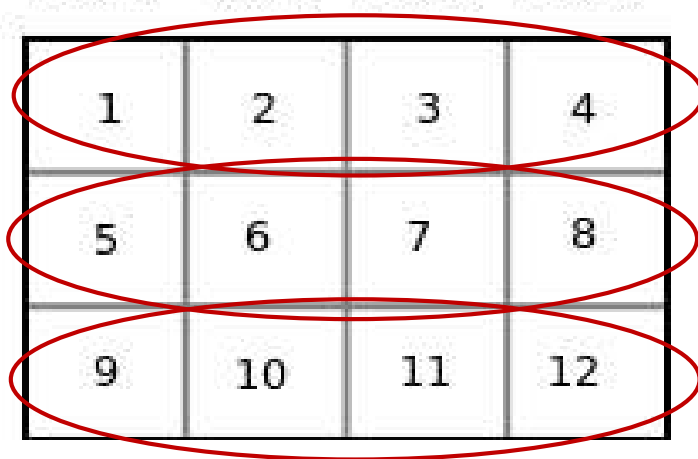
*$ptr = arr$
same as: $ptr = \&arr[0]$*



Pointer to multidimensional arrays

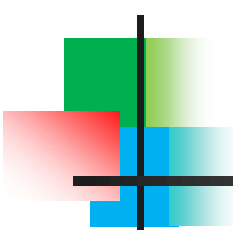
```
int arr[3][4]= {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
```

	Col 1	Col 2	Col 3	Col 4
Row 1	1	2	3	4
Row 2	5	6	7	8
Row 3	9	10	11	12



What is the difference between arr and &arr[0][0]





2-D arrays

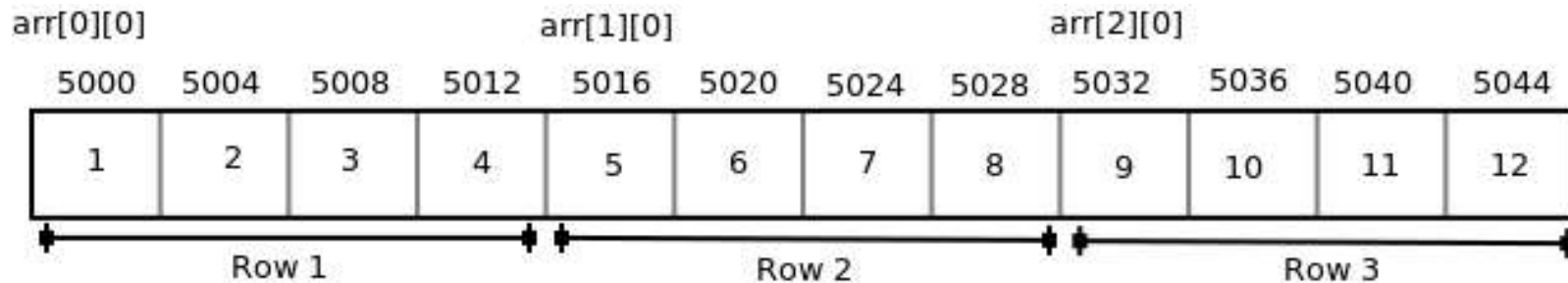
- A 2-D array is a 1-D array, where each element is a 1-D array
 - `arr[3][4]` is a 1-D array of size 3, where each element of `arr[3][4]` is a 1-D array of size 4 (i.e. 4 integers)
 - Name of a 1-D array is a constant pointer to the 0th element
 - If `arr = 5000`, then `arr + 1` is equal to
$$\text{arr} + 1 * \text{sizeof}(1\text{-D array}) = 5000 + 1 \times 16 = 5016$$
 - $$\text{arr} + 2 = 5000 + 2 \times 16 = 5032$$

*The first element of `arr` is not an integer; it's the entire first row (`arr[0]`, which is an array of 4 integers). The size of one row (a "1-D array" of 4 integers) is $4 * \text{sizeof}(\text{int})$, which is $4 * 4 = 16$ bytes.*



Pointer to multidimensional arrays

- `int arr[3][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};`
- Memory is linear
 - Not possible to store 2D array in terms of rows and columns

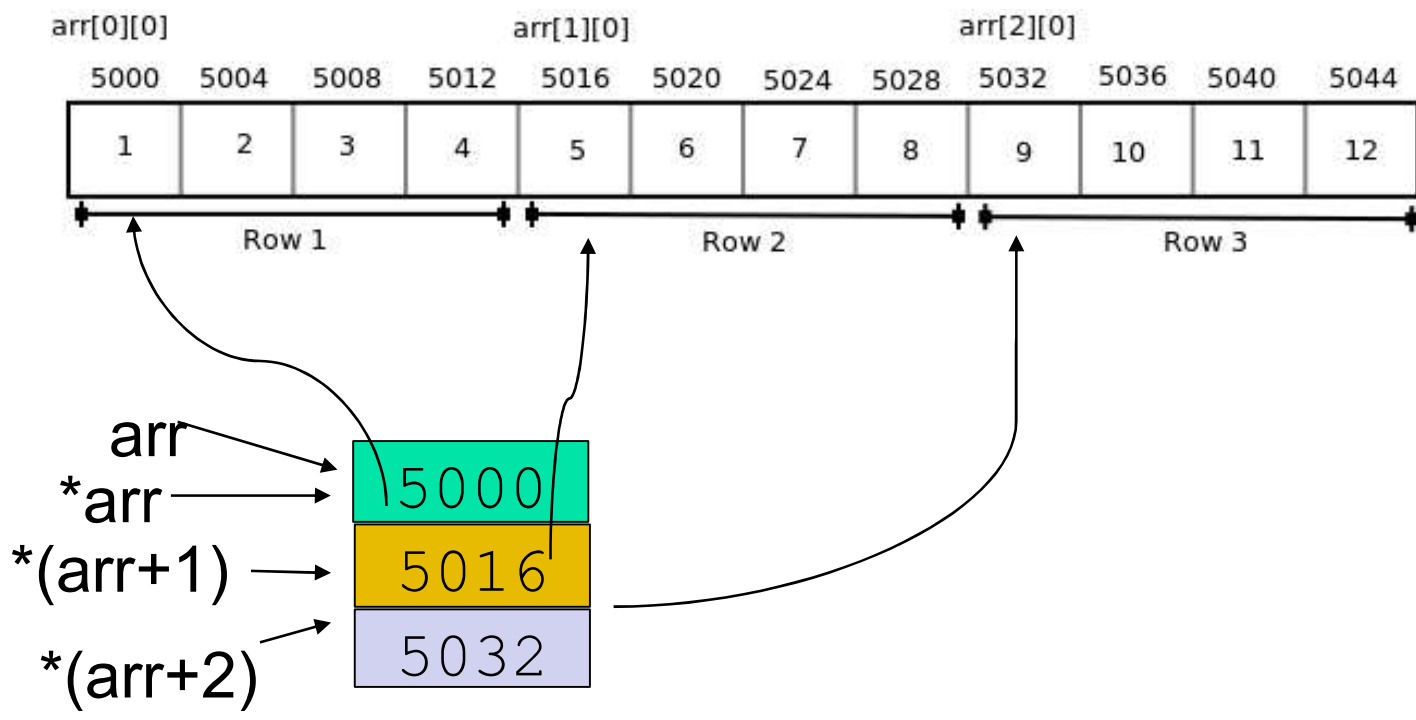
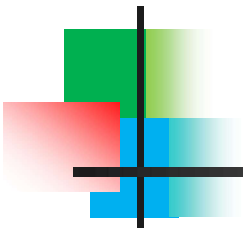




Dereferencing pointer to array names

- Dereferencing pointer to an array gives the base address of the array
 - On dereferencing arr, we get *arr.
 - Note that both arr and *arr points to the same address
 - But base type of arr is a pointer to 4 integers but base type of *arr is integer.

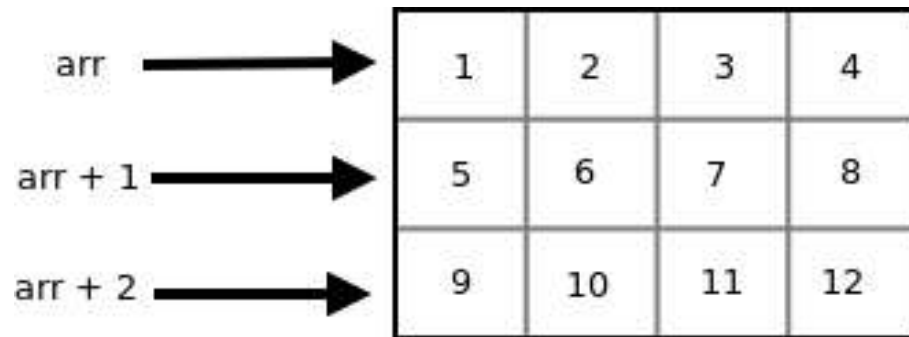




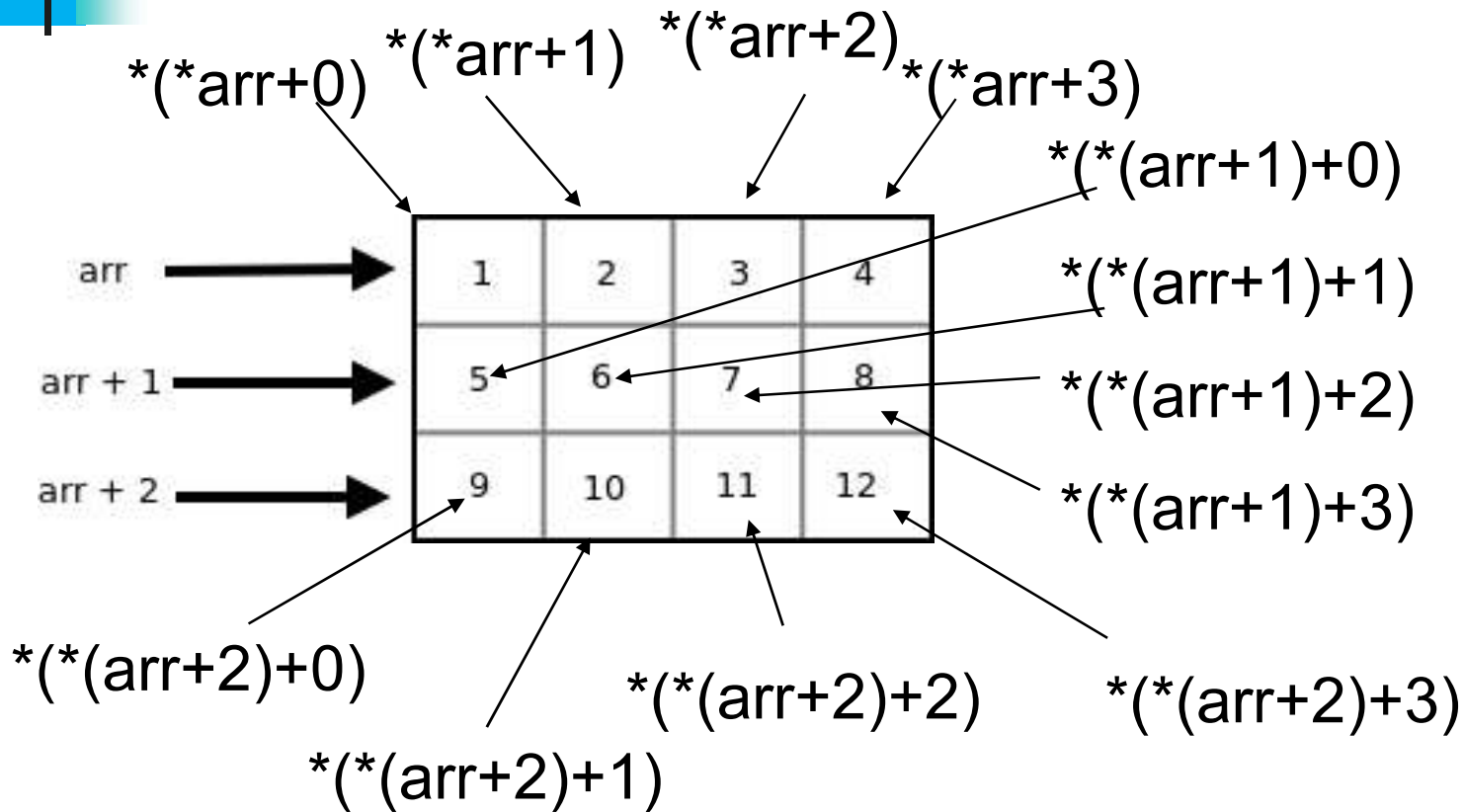


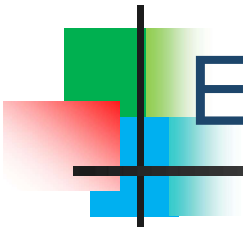
Accessing 2D array elements

- Accessing the array elements of a 2D array



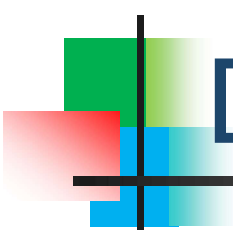
Another view of 2-D array





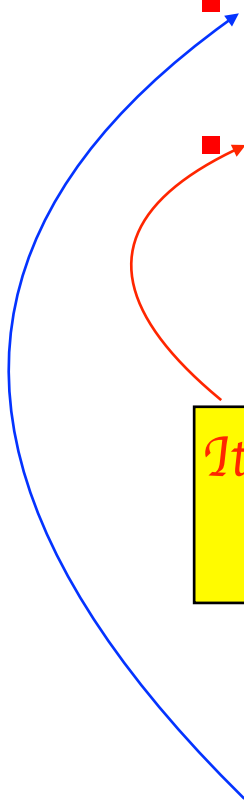
Ex:Printing elements of a 2D array

- Write a program to print the elements of the 2-D array `arr[3][4]` described earlier.



Difference Arrays and Pointers

- **double A[10];** vs. **double *A;**
- Only difference:—
 - **double A[10]** sets aside *ten* units of memory, each large enough to hold a **double**
 - **double *A** sets aside *one* pointer-sized unit of memory
 - You are expected to come up with the memory elsewhere!



It tells the compiler, "Please reserve a small piece of memory (e.g., 8 bytes) that is just big enough to hold one memory address."



It tells the compiler, "Please reserve a block of memory for me that is large enough to hold 10 double values all in a row."